Development of a Large Language Model-Based

Interface for Robotics Systems using UR5E

Final Year Project — Working Overview Document

Nanyang Technological University, School of EEE | Year 4 | Supervisor: Prof Cheah
FYP period: August 2026 – May 2027 | Preparation period: 16 June 2026 – 4 August 2026 (6.5 weeks, self-directed)

---

1. Project Overview

The project develops and compares two architectures for natural-language-driven robotic pick-and-place manipulation on a UR5e arm, using generalisation to unseen objects as the central evaluation axis. The core research question is whether a modular pipeline that pairs an LLM planner with open-vocabulary perception generalises better (or worse, or differently) than an end-to-end Vision-Language-Action (VLA) model fine-tuned on real demonstrations.

Architecture A — Modular Pipeline

- LLM planner (Anthropic API via OpenRouter) sequences high-level actions as structured tool calls
- Open-vocabulary object detection (GroundingDINO or OWL-ViT) — not a closed-set detector, so it can name and localise novel object categories from free-form text
- Depth-to-3D localisation: 2D pixel box centre + RGB-D depth + camera intrinsics → camera-frame XYZ
- Hand-eye (eye-to-hand) calibration: static camera-to-base transform chains camera-frame points into the robot base frame
- Scripted motion primitives (pick / place) built on top of the URController class
- Closed-loop re-perception: the system re-observes the scene between steps rather than executing one open-loop plan blindly

Architecture B — End-to-End VLA

- SmolVLA fine-tuned directly on real UR5e teleoperation demonstrations
- Demonstrations captured via a custom DemoRecorder: synchronised joint positions, TCP pose, gripper state, and camera image, timestamped at a fixed rate (e.g. 20 Hz)
- Constraints and behaviour are learned statistically from demonstration data rather than injected explicitly via a system prompt

Why this comparison matters

The two architectures trade off differently. The modular pipeline (A) offers localised failure diagnosis — if something goes wrong, it is usually attributable to a specific stage (detection, depth conversion, calibration, or planning) — and allows explicit constraint injection through the LLM system prompt. The end-to-end VLA (B) encodes constraints implicitly and statistically through its training demonstrations, which is harder to debug but potentially more fluid in its motion generation. Testing both against the same generalisation benchmark is intended to make this trade-off empirically visible rather than asserted.

2. Evaluation Design

Generalisation to unseen objects is evaluated across three escalating tiers:

Tier	Task	What it tests
Tier A	Pick-and-place	Baseline competence on trained / seen object categories
Tier B	Category sorting	Generalisation within a known category to new instances
Tier C	Novel-object generalisation probe	Generalisation to object categories never seen in training or demonstrations — the project's key claim, echoing RT-2's headline generalisation result

3. Hardware & Software Stack

Category	Details
Robot	UR5e arm, fixed RGB-D camera (eye-to-hand configuration), parallel-jaw gripper
Compute (local)	Lenovo Legion Pro 5i Gen 10 — RTX 5070 Ti (12GB), Ubuntu 24.04
Compute (cloud, training)	Modal Labs or Colab Pro, for VLA fine-tuning
Compute (cloud, inference)	Anthropic API / OpenRouter, for the LLM planner
Robot control	ur_rtde (RTDEControlInterface for commands, RTDEReceiveInterface for state reads), URSim 5.25.2 (native Linux install)
Perception	GroundingDINO / OWL-ViT (open-vocabulary detection)
VLA / imitation learning	SmolVLA, LeRobot, ManiSkill3 (simulation, used for prep/learning only — UR5e in ManiSkill3 with a fixed third-person camera)
Dev environment	Ubuntu 24.04, uv (Python package management, uv.lock committed), VS Code, Git/GitHub over SSH
Repo layout	src/fyp/ (importable package), scripts/ (exploratory), data/, tests/, docs/

4. Background & Prior Experience

JJ enters this project with hands-on experience from a prior Tank project involving micro-ROS, SLAM and Nav2, TF, and odometry — giving a solid working grasp of coordinate frames and core ROS concepts. UR robots and arm kinematics specifically (FK/IK, singularities, UR-specific pose conventions, Polyscope/URScript) were new at the start of the preparation period.

5. Preparation Period Progress (16 Jun – 4 Aug 2026)

The 6.5-week self-directed prep period runs at roughly 15 hours/week ahead of the formal FYP start, and was revised mid-way to align with the supervisor-agreed scope: the modular-vs-VLA comparison, generalisation to unseen objects as the evaluation focus, demonstrations collected directly on the physical UR5e (rather than from egocentric hand-pose video), and a closed-loop control strategy.

Week	Dates	Focus
1	16–22 Jun	UR fundamentals + Polyscope basics (e-Series Core Track)
2	23–29 Jun	URScript language + kinematics (FK, IK, rotations, frames)
3	30 Jun – 6 Jul	Python-to-UR via ur_rtde; build the URController class
4	7–13 Jul	VLA paper reading (RT-2, OpenVLA, OXE) + ManiSkill3 simulation
5	14–20 Jul	SmolVLA + LeRobot reproduction + RTC paper
6	21–27 Jul	Perception + modular pipeline + LLM planner (closed-loop slice)
6.5	28 Jul – 4 Aug	Proposal refinement, consolidation, supervisor sign-off

Status: currently mid Week 3 – Week 4 boundary

Completed:

- URSim 5.25.2 installed natively on Ubuntu 24.04 (patched for compatibility; an earlier Docker approach was superseded)
- Git/GitHub set up with SSH, .gitignore, and a uv-managed Python environment
- ur_rtde validated end-to-end: both RTDEControlInterface and RTDEReceiveInterface working
- Pose transform pipeline built from scratch in NumPy — pose_to_T, T_to_pose, pose_trans — using the Rodrigues rotation formula
- RT-1 and RT-2 papers read with deep architectural walkthroughs
- Repository restructured into src/fyp/, scripts/, data/, tests/, docs/

Immediate next steps:

- OpenVLA paper — continuing the VLA reading sequence (RT-1 → RT-2 → OpenVLA)
- Week 4: build the DemoRecorder class (synchronised joint positions, TCP pose, gripper state, camera image at a fixed rate)

On the horizon:

- Week 4: ManiSkill3 setup with UR5e and a fixed camera; DemoRecorder built and validated
- Week 5: SmolVLA + LeRobot reproduction; RTC (real-time control) paper
- Week 6: perception / modular-pipeline thread — open-vocabulary detection, depth-to-3D, hand-eye calibration, scripted primitives, Anthropic SDK tool-calling, closed-loop slice in simulation
- Week 6.5: proposal refinement, codebase consolidation, supervisor sign-off

- Anthropic API integration via OpenRouter, potentially connected to an Obsidian vault for project context continuity

6. Key Technical Learnings & Principles

- **Architecture trade-off:** Architecture A gives localised failure diagnosis and explicit constraint injection via the system prompt; Architecture B encodes constraints statistically through demonstrations, which is more fluid but harder to debug.
- **RT-1 / RT-2 action representation:** discretised action bins are still discrete tokens, not continuous values, and action tokens are model outputs — not fed back in as previous-state inputs.
- **Pose composition:** T_new = T_prev @ T_offset requires the full homogeneous transform, not just the rotation matrix; axis-angle extraction has a singularity near $\theta \approx \pi$ that needs explicit handling.
- **ur_rtde interfaces:** RTDEControlInterface (commands) and RTDEReceiveInterface (state reads) are separate objects, not one unified interface.
- **moveJ vs moveJ_IK:** moveJ interprets its argument as joint angles; moveJ_IK takes a Cartesian pose and solves IK internally.
- **Platform:** the stack runs on Linux or Windows; Linux is preferred for fewer papercuts, not a hard requirement.
- **Frame fluency carries over:** pose_trans (compose two frames) and pose_inv (invert a transform) map directly onto the TF concepts used in the prior Tank project, and this fluency feeds directly into Week 6 hand-eye calibration.

7. Working Style & Preferences (for this workspace)

- Learns by hand-typing code and asking targeted, single-concept questions rather than reading complete implementations end to end
- Prefers conceptual grounding before code: why → how → where, with physical analogies before formulas
- Sequential, structured teaching — one component at a time, explicit stopping points, confirmation before moving on
- Will explicitly ask to slow down or restart when an explanation gets too dense
- Prefers template code to edit and extend, rather than complete implementations handed over wholesale
- Expects direct, accurate technical pushback — precise about "smoother on X" vs "requires X", and flags when a figure is an empirical estimate rather than an exact one
- Keeps a weekly structure with daily checkboxes; papers are often read during commutes

8. Tools & References

Category	Items
Robot control	ur_rtde, URSim 5.25.2 (native Linux), UR5e hardware
ML / robotics stack	PyTorch, CUDA, GroundingDINO, SmolVLA / LeRobot, ManiSkill3
Dev environment	Ubuntu 24.04, uv, VS Code, GitHub (SSH), uv.lock committed
Key references	UR Script Manual (SW5.11); Lynch, Modern Robotics (Ch. 2–4, 6); RT-1, RT-2, OpenVLA, Open X-Embodiment, SmolVLA, RTC papers
Background-only reading	EgoMimic, UMI (lit review context, not hands-on)
Project path	~/Documents/NTU/Y4S1/FYP/

Generated from the working context of this Claude project space. Reflects progress and plans as discussed up to 3 July 2026; treat weekly dates/status as a snapshot, not a live tracker.

User Profile — JJ

A simple reference profile summarising who JJ is in the context of this project workspace.

Academic

- Year 4 undergraduate, School of Electrical and Electronic Engineering (EEE), NTU
- Final Year Project: "Development of a Large Language Model-Based Interface for Robotics Systems using UR5E", supervised by Prof Cheah
- FYP timeline: August 2026 – May 2027, preceded by a self-directed 6.5-week preparation period starting mid-June 2026

Technical background

- Prior robotics project experience: a Tank project using micro-ROS, SLAM/Nav2, TF, and odometry
- Comfortable with coordinate frames and general ROS concepts coming in
- New to UR robots, arm kinematics, and manipulator-specific tooling (Polyscope/URScript, ur_rtde) at the start of this project — has since built working fluency through the prep plan
- Comfortable working across Python and native Linux tooling; uses uv for environment management and Git/GitHub with SSH

Learning & working style

- Learns hands-on: hand-types code, asks narrow single-concept questions rather than absorbing full implementations
- Wants conceptual grounding (why/how/where, physical analogies) before formulas or code
- Works best with sequential, one-piece-at-a-time explanations with explicit checkpoints
- Comfortable pushing back on imprecise or overstated technical claims, and expects the same rigour in return
- Plans work in structured weekly blocks with daily checkboxes; fits reading around a commute

Communication preference

- Prefers concise, focused answers over long-form explanation by default; expands only when asked